Guião Laboratorial

Aula 04 — Descoberta e Interrogação de Dispositivos BLE

Tecnologias de Comunicação para a Internet das Coisas

Ano letivo 2026–2027

Catarina Silva

ESTGA, Universidade de Aveiro

Duração prevista: 2 horas

Índice

Enquadramento **2**

Modos de Execução: com ou sem Hardware **2**

Objetivos de Aprendizagem **3**

1 Preparação do Adaptador **4**

2 Descoberta de Dispositivos **5**

3 Interrogação da Hierarquia GATT **6**

4 Anúncio e Ligação **7**

5 Análise de uma Captura HCI (opcional, 20 min) **9**

Questões para Discussão **9**

Referências **9**

Enquadramento

A componente teórica descreveu a camada de ligação, os estados de anúncio e varrimento, e a organização dos dados em serviços e características segundo o GATT. Esta sessão observa esses mecanismos em funcionamento, sobre dispositivos reais, recorrendo às ferramentas da pilha BlueZ apresentadas nos diapositivos.

Não se desenvolve software. Os comandos utilizados são fornecidos e executados numa consola, e o trabalho do estudante consiste em interpretar o que é devolvido e em relacioná-lo com os conceitos da aula teórica.

Material

- Raspberry Pi com BlueZ instalado, no papel de dispositivo central.
- Um smartphone por grupo, com a aplicação nRF Connect instalada, no papel de periférico BLE. Não é necessário qualquer equipamento adicional.
- Sem este equipamento, seguir o modo simulado descrito em "Modos de Execução".

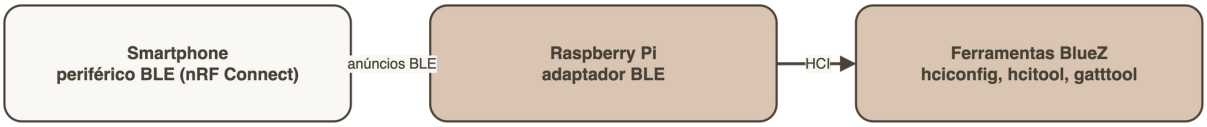


Figura 1: *Arquitetura do trabalho*

Modos de Execução: com ou sem Hardware

O guião pode ser feito em três modos. Os comandos e a interpretação dos resultados são os mesmos; só muda a origem dos dispositivos.

Modo	Equipamento	Como se faz
A. Físico	Raspberry Pi + smartphone com nRF Connect	Comandos reais; "telemóvel" é o smartphone
B. Misto	Só o smartphone, ou só o Pi	Parte real e parte simulada (por exemplo, o Pi varre mas o periférico é simulado)
C. Simulado	Qualquer computador com Python 3	`ble_sim.py` imita as ferramentas; "telemóvel" é o comando `phone adv ...`

Preparar o modo simulado

Descarregar o ficheiro `codigo.zip` (botão *Source Code* da aula na página da UC), extraí-lo e, na pasta extraída, executar:

```
python3 ble_sim.py        # no Windows: python ble_sim.py
```

Não há dependências: basta Python 3.8 ou superior em Linux, macOS ou Windows. O simulador abre uma consola que aceita `hciconfig`, `sudo hcitool lescan` e `gatttool -b <endereço> [-t random] -I`, com as mesmas opções e um formato de saída muito próximo do real. O adaptador simulado começa *down*, tal como num Pi recém-ligado.

Dispositivos simulados

- `TCIoT-Sensor` (endereço público): conectável; serviços Generic Access, Battery (com notificações) e um serviço do fabricante com temperatura em centésimos de grau.
- `TCIoT-Beacon`: anunciante **não** conectável, com dados no próprio anúncio.
- `Band-Mock`: pulseira com endereço **aleatório** que muda a cada varrimento; o nome só chega na resposta a varrimento.
- `Telemovel-Aluno`: substitui o nRF Connect. Controla-se com `phone adv on`, `phone adv off`, `phone adv conn` e `phone adv nonconn`.

Instalar as ferramentas de análise (modo B/C e secção 5)

- **Wireshark** (Windows, macOS e Linux): `https://www.wireshark.org/download.html`; em Debian/Ubuntu, `sudo apt install wireshark`.
- **Bumble** (opcional, pilha Bluetooth em Python com controladores virtuais): `python3 -m venv venv && venv/bin/pip install bumble`. Permite, por exemplo, `venv/bin/bumble-show -format snoop captura_ble.btsnoop`.

Alternativas avançadas (não necessárias)

Um emulador Android recente inclui um controlador Bluetooth virtual; em Linux, os módulos `hci_vhci` e o emulador de testes do BlueZ criam adaptadores virtuais que o `bluetoothctl` reconhece. Dependem do sistema anfitrião e não foram validados para esta aula; usar apenas como projeto de exploração.

Objetivos de Aprendizagem

- Identificar dispositivos BLE na vizinhança e distinguir os que se anunciam de forma conectável dos que apenas difundem informação;
- Percorrer a hierarquia GATT de um dispositivo real, do serviço até ao valor da característica;
- Relacionar os identificadores observados (UUID e *handle*) com os conceitos de tipo e de endereço de atributo apresentados na teoria;
- Reconhecer, na prática, a diferença entre comunicação por anúncio e comunicação por ligação.

1. Preparação do Adaptador

15 min

Objetivo

Confirmar a presença do adaptador BLE e colocá-lo em estado operacional.

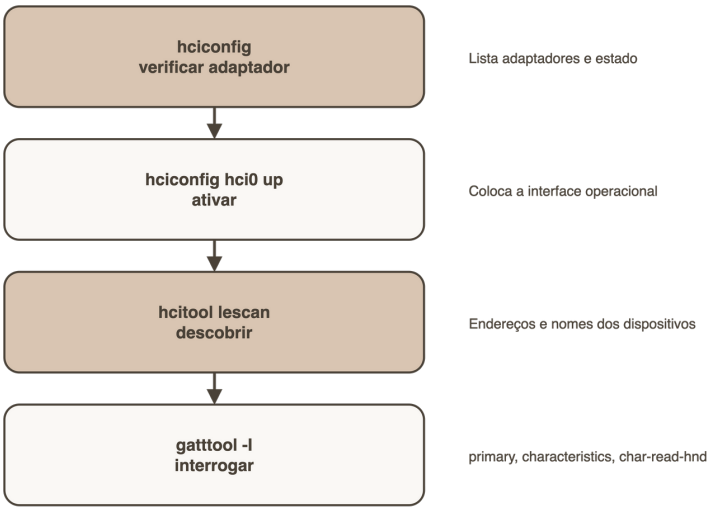


Figura 2: *Sequência de utilização das ferramentas*

```
hciconfig
sudo hciconfig hci0 up
hciconfig -a hci0
```

1. Executar `hciconfig` sem argumentos e registar o nome da interface apresentada, habitualmente `hci0`, bem como o respetivo endereço.
2. Ativar a interface e voltar a consultar o estado com a opção `-a`.
3. Registar a versão de HCI indicada e confrontá-la com a versão da especificação Bluetooth correspondente.

Questão

O endereço apresentado pelo `hciconfig` é público ou aleatório? Que consequência tem esta distinção para a possibilidade de rastrear o dispositivo ao longo do tempo?

Indicação

Os endereços de adaptador são, em geral, públicos. Os de um smartphone moderno quase nunca o são: ver a secção 2.

Configuração do BlueZ (opcional, Raspberry Pi)

Se a interface `hci0` não surgir ou o Pi for partilhado por vários grupos:

```
sudo nano /etc/hostname      # nome distinto por grupo
sudo apt install bluez bluez-hcidump
sudo nano /lib/systemd/system/bluetooth.service
#  ExecStart=/usr/lib/bluetooth/bluetoothd --experimental
```

```
sudo nano /etc/bluetooth/main.conf     # ControllerMode = le
sudo systemctl daemon-reload
sudo systemctl restart bluetooth
```

O modo `le` deixa o adaptador a operar só em BLE, o que simplifica os varrimentos. Em simulação, saltar este passo.

2. Descoberta de Dispositivos

30 min

Objetivo

Observar o tráfego de anúncio e caracterizar os dispositivos presentes na sala.

```
sudo hcitool lescan
```

1. Executar o varrimento durante cerca de 30 segundos e interromper com `Ctrl+C`.
2. Registar numa tabela os endereços e nomes obtidos. Anotar quantos dispositivos apresentam nome e quantos surgem apenas com endereço.
3. No smartphone, abrir o nRF Connect e ativar o modo periférico (*Advertiser*), com um nome identificável pelo grupo. Repetir o varrimento e confirmar que o dispositivo passa a constar da lista.
4. Desativar o anúncio no smartphone e repetir o varrimento uma terceira vez.

Endereço	Nome	Observações

Endereços aleatórios

Os endereços de smartphones são, normalmente, **privados resolúveis**: mudam com o tempo. Por isso, ao repetir o varrimento, o telemóvel pode surgir com outro endereço; é o endereço mais recente que deve ser usado nos passos seguintes. Uma pulseira que mantivesse o endereço seria rastreável; uma que o muda deixa de o ser (ver a teoria).

Registar também: um dispositivo que apareça com `(unknown)` e, logo a seguir, com nome corresponde a um anúncio sem nome seguido da resposta a varrimento ativo.

Relação com a teoria

O varrimento realizado pelo `hcitool lescan` é ativo: além de escutar os canais 37, 38 e 39, emite pedidos de varrimento aos quais o anunciante responde com informação adicional. É por esse motivo que alguns dispositivos apresentam nome e outros não.

Alternativa com o bluetoothctl

O `hcitool` está *deprecated*. Com a ferramenta atual do BlueZ:

```
bluetoothctl
[bluetooth]# scan on
[bluetooth]# scan off
```

Além de endereço e nome, mostra o RSSI, a potência de emissão e os dados do anúncio, o que permite medir a distância aproximada com o modelo da teoria. Em modo simulado, usar `hcitool lescan`.

3. Interrogação da Hierarquia GATT

45 min

Objetivo

Percorrer os serviços e características de um periférico real e ler o valor de uma característica.

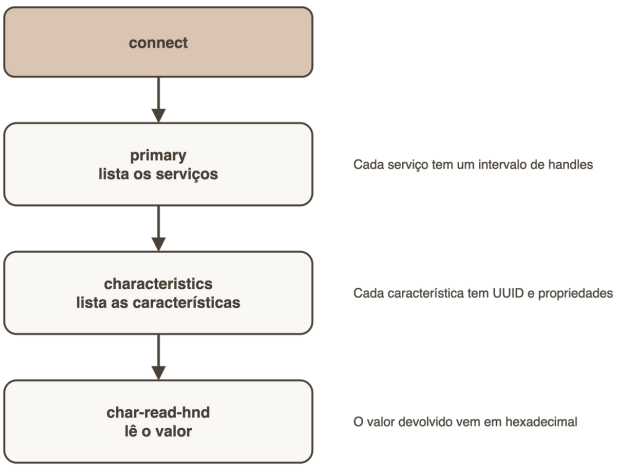


Figura 3: *Sequência de exploração GATT*

Manter o smartphone a anunciar-se e, a partir do Raspberry Pi, executar:

```
sudo gatttool -I -b <ENDERECO_DO_DISPOSITIVO>
```

No modo interativo, emitir sucessivamente:

```
connect
primary
characteristics
char-read-hnd <handle>
disconnect
```

1. Registar os serviços devolvidos por `primary`, anotando para cada um o UUID e o intervalo de *handles*.
2. Registar as características devolvidas por `characteristics`, anotando o *handle* do valor, as propriedades e o UUID.
3. Escolher uma característica cujas propriedades incluam leitura e obter o seu valor com `char-read-hnd`. O resultado é apresentado em hexadecimal.

4. Identificar, entre os UUID observados, os que correspondem a serviços normalizados pelo Bluetooth SIG e os que são específicos do fabricante.

Endereços aleatórios e a opção -t random

Se o endereço for aleatório (caso dos smartphones e da `Band-Mock`), o `gatttool` precisa de o saber: `sudo gatttool -b <endereço> -t random -I`. Sem esta opção, a ligação falha com *Connection refused*, e a falha não indica problema no dispositivo.

5. Listar todos os atributos com `char-desc`. Identificar, pelos UUID `0x2800`, `0x2803` e `0x2902`, as declarações de serviço, as declarações de característica e os descritores de configuração de notificação. Comparar a organização com a figura da hierarquia GATT.
6. Ler a característica do nível de bateria (UUID `0x2A19`) e converter o resultado de hexadecimal para decimal. Por exemplo, `57` corresponde a 87 %.
7. Ativar notificações: escrever `0100` no descritor `0x2902` da bateria, com `char-write-req <handle do descritor> 0100`. Observar as linhas `Notification handle = ...` que surgem sem novo pedido, e desativar escrevendo `0000`.
8. Ler a característica do fabricante que guarda a temperatura (dois bytes, *little-endian*, em centésimos de grau) e calcular o valor em °C.

Questão

Qual a vantagem de usar notificações em vez de repetir `char-read-hnd`? Em que situações é preferível uma indicação?

Leitura dos identificadores

Os UUID de 16 bits, apresentados na forma `0x180A`, correspondem a serviços adotados pelo Bluetooth SIG. Os UUID de 128 bits identificam serviços definidos pelo fabricante. Esta distinção permite reconhecer de imediato que parte do dispositivo segue perfis normalizados.

Trabalho complementar

Com a aplicação nRF Connect no papel de cliente, ligar-se a um segundo dispositivo e comparar a informação apresentada pela aplicação com a devolvida pelo `gatttool`. A aplicação resolve automaticamente os UUID normalizados para nomes legíveis; identificar quais.

4. Anúncio e Ligação

20 min

Objetivo

Distinguir experimentalmente os dois paradigmas de comunicação estudados.

1. Configurar o nRF Connect para difundir dados no próprio pacote de anúncio, sem permitir ligação.
2. Confirmar, por varrimento, que o dispositivo continua visível e que os dados são legíveis sem qualquer ligação estabelecida.
3. Tentar ligar com o `gatttool` e registar o resultado.

4. Repor o modo conectável e confirmar que a ligação passa a ser possível.

Modo simulado

Em vez do nRF Connect, usar `phone adv nonconn`, repetir `sudo hcitool lescan` para confirmar que o telemóvel continua visível e tentar `gatttool -b <endereço> -t random -I` seguido de `connect`. Repor com `phone adv conn`. O endereço muda ao reconfigurar o anúncio: ler o novo valor no varrimento.

5. Análise de uma Captura HCI (opcional, 20 min)

Alternativa ou complemento ao varrimento ao vivo

Objetivo

Reconhecer anúncios, ligação e transações GATT num ficheiro de captura, como o que o Android produz com o *HCI snoop log*.

O ficheiro `captura_ble.btsnoop` (incluído no `codigo.zip`) contém 26 pacotes de uma sessão equivalente à das secções 2 e 3. Abrir o ficheiro no Wireshark (*File* → *Open*) e usar os filtros `bthci_evt` (eventos HCI) e `btatt` (protocolo de atributos).

1. Quantos anunciantes distintos aparecem? Quais são conectáveis e qual não é? O tipo de evento do relatório de anúncio indica-o (`ADV_IND`, `ADV_NONCONN_IND`, `SCAN_RSP`).
2. Qual o endereço cujo tipo é aleatório e porque é que o seu nome vem num pacote separado?
3. Identificar o pedido de ligação, o intervalo de ligação acordado (em unidades de 1,25 ms) e o *handle* da ligação.
4. Filtrar por `btatt.opcode == 0x1b` e registar o *handle* e o valor de cada notificação. Que escrita anterior as ativou?
5. Para o pedido de leitura de bateria, converter o valor devolvido para percentagem.

Em alternativa, sem Wireshark: `bumble-show -format snoop captura_ble.btsnoop`. Para produzir uma captura real, ativar o *Bluetooth HCI snoop log* no Android (opções de programador) ou correr `sudo btmon -w captura.btsnoop` no Raspberry Pi durante os exercícios anteriores.

Questões para Discussão

1. Que informação foi possível obter sobre dispositivos de terceiros sem qualquer autorização, e que implicações tem esse facto?
2. Por que razão a difusão por anúncio é mais económica em energia do que a comunicação por ligação?
3. O caso das pulseiras de atividade apresentado na teoria é observável na sala? Como se poderia verificar?
4. Qual o método de emparelhamento que seria utilizado entre o Raspberry Pi e um sensor sem visor nem teclado, e que risco isso acarreta?
5. Compare as ferramentas `hcitool`/`gatttool` com o `bluetoothctl`: porque razão o BlueZ as marcou como *deprecated*, e que riscos há em depender delas?
6. Um sensor de temperatura envia uma leitura por minuto. Justifique se deve usar comunicação por anúncio ou por ligação com notificações, tendo em conta energia, fiabilidade e privacidade.

Referências

- Gonçalves, P. *Apresent4 — Bluetooth*, TCIoT, edições anteriores.
- Bluetooth SIG. *Bluetooth Core Specification*.
- Documentação do `hciconfig`: `https://helpmanual.io/man1/hciconfig/`
- Documentação do `gatttool`: `https://helpmanual.io/man1/gatttool/`
- BlueZ. *bluetoothctl* e *btmon*: `https://www.bluez.org/`

- Wireshark, *Bluetooth*: `https://wiki.wireshark.org/Bluetooth`
- Bumble, pilha Bluetooth em Python: `https://google.github.io/bumble/`